

Capstone Project: Smart Certificate Verification & Student Records Management System

Project Overview

Educational institutions often need a secure system to verify student certificates and manage academic records. Many employers and graduates need quick verification of certificates, transcripts, and student achievements.

In this project, you will design and develop a Smart Certificate Verification and Student Records Management System that allows:

- Students to verify certificates
- Administrators to manage student records
- Employers to validate certificates
- System to store academic records in a database
- Generate reports
- Handle file storage for certificates/transcripts

This project combines:

- Object-Oriented Design (UML)
- C# Programming
- Database Integration
- File Handling
- Exception Handling
- Inheritance

- Searching/Sorting
- GUI/Event Handling (Optional if using Windows Forms/WPF)

Business Scenario

SkillsFuture Singapore and private educational institutions need a secure digital verification platform where graduates can verify certificates and employers can validate academic credentials.

The institution currently faces problems:

- Manual certificate verification process
- Lost student records
- Slow transcript retrieval
- Fake certificate submissions
- No centralized digital platform

Your team has been hired to build a solution.

Project Objectives

Students must:

1. Design the system using UML diagrams
2. Apply object-oriented design principles
3. Develop a working C# application
4. Integrate database functionalities
5. Implement file processing
6. Demonstrate inheritance and polymorphism
7. Apply searching and sorting algorithms

8. Handle exceptions properly
9. Present the completed system

System Features

Your system must include the following modules:

1. User Management Module

Users can register/login based on roles:

- Student
- Admin
- Employer

Functionalities:

- Register user
- Login authentication
- Logout
- Role assignment

2. Certificate Verification Module

Students or employers can verify certificates using:

- Certificate ID
- Student Name
- Date of Birth

Output:

- Certificate validity
- Award title
- Completion date
- Transcript access

3. Student Records Management

Admin can:

- Add student
- Update student details
- Delete student records
- View all students

4. Course Management Module

Admin can:

- Add courses
- Assign modules
- View enrolled students

5. Transcript Management

System should:

- Generate transcript

- Calculate GPA
- Display grades

6. File Management Module

Store and retrieve:

- PDF certificates
- Transcript files
- Student documents

Use:

- File streams
- Directories
- File handling

7. Search and Sorting Module

Implement:

- Search student by name
- Search certificate by ID
- Sort students by GPA
- Sort students alphabetically

Use arrays or collections.

8. Database Module

Use:

- SQL Server
- OR
- SQLite
- OR
- PostgreSQL

Required:

- CRUD operations
- SQL queries
- LINQ queries
- LINQ operators

Part A: Object-Oriented Design Deliverables (30%)

Students must design the following UML diagrams before coding.

1. Use Case Diagram

Actors:

- Student
- Employer
- Admin

Include use cases such as:

- Login
- Verify Certificate
- Manage Student Records
- Generate Transcript

2. Class Diagram

Must include:

- Classes
- Attributes
- Methods
- Associations
- Multiplicity
- Aggregation
- Generalization

Example classes:

- User
- Student
- Admin
- Employer
- Certificate
- Transcript

- Course
- DatabaseManager

3. Sequence Diagram

Scenario:

Employer verifies a certificate

Flow:

1. Enter certificate details
2. System validates input
3. Database checks record
4. Result displayed

4. Activity Diagram

Process flow for:

- Student registration
- Certificate verification

5. Component Diagram

Show:

- UI layer
- Business logic layer
- Database layer
- File management layer

6. Package Diagram

Group related classes:

- Models
- Services
- Database
- Utilities
- UI

Part B: C# Development Deliverables (50%)

Develop the actual system.

Required OOP Features

Classes and Objects

Create multiple classes.

Example:

Student

Certificate

Transcript

Admin

Employer

Method Overloading

Example:

VerifyCertificate(int certID)

VerifyCertificate(string certID, string studentName)

Inheritance

Example:

User

|

| | |

Student Admin Employer

Exception Handling

Handle:

- Invalid login
- Missing files
- Database connection issues
- Invalid certificate input

Example:

try

{

 //code

}

catch(Exception ex)

{

 Console.WriteLine(ex.Message);

}

Arrays/Search/Sort

Example:

- Sort students by GPA
- Search by certificate ID

File Handling

Use:

- StreamReader
- StreamWriter
- FileStream

Store certificates/transcripts.

Database Integration

Must demonstrate:

- SQL queries
- LINQ queries
- CRUD operations

Part C: User Interface (Optional Bonus 10%)

Students may choose:

- Console App
- Windows Forms
- WPF
- ASP.NET MVC

Advanced students may build a web-based version using ASP.NET Core.

Project Development Phases

Phase 1: Requirement Analysis

Deliverables:

- Problem statement
- Functional requirements
- Non-functional requirements

Phase 2: System Design

Deliverables:

- UML diagrams
- Database schema

Phase 3: Development

Deliverables:

- Working source code

Phase 4: Testing

Perform:

- Unit testing
- Functional testing
- Exception testing

Phase 5: Final Presentation

Demonstrate:

- UML diagrams
- Working application
- Challenges encountered
- Future improvements

Final Submission Requirements

Submit:

1. Source code
2. UML diagrams
3. Database scripts
4. Test cases
5. Project documentation
6. Presentation slides
7. Demo video of the system

Marking Rubrics

Criteria	Weight
UML Design	30%
C# Functionality	30%
Database Integration	15%
File Handling	10%
OOP Concepts	10%
Presentation	5%

Final Expected Outcome

By completing this project, students will demonstrate mastery of:

- Object-oriented design
- UML modelling
- C# programming
- Database development
- Real-world problem solving